El algoritmo de Dijkstra

Teoría de Grafos y Caminos Mínimos

Grupo 3

Universidad de Córdoba

19 de abril de 2026

Integrantes del grupo

Alijo Fernández, Miguel Ángel
Aramyan, David Ángel
Buzón García, Alejandro
Casado Caballero, Antonio
Collantes Aguilar, Pablo
Costa Ruiz-Canela, Javier Jesús
Deogracias Moya, Juan
Guerrero Romero, Jose Francisco
Laredo Gutiérrez, José Raúl
Ligero González, Alejandro
López Reyes, Marcos

UNIVERSIDAD DE CÓRDOBA

Tabla de contenidos

1. Introducción
2. Descripción del algoritmo
3. Fundamento matemático
4. Ejemplo paso a paso
5. Pseudocódigo
6. Simulación y Conclusiones

UNIVERSIDAD DE CÓRDOBA

Introducción

Pregunta inicial

Si existen varios caminos para llegar de un punto a otro, ¿cómo podemos determinar cuál es el más eficiente?

- En muchos problemas no basta con llegar al destino.
- También interesa minimizar la distancia, el tiempo, el coste o los recursos.
- Este desafío se conoce como el **problema del camino mínimo**.

El algoritmo de Dijkstra permite abordar este problema en grafos ponderados con pesos no negativos.

UNIVERSIDAD DE CÓRDOBA

Motivación del problema

El problema del camino mínimo aparece en multitud de situaciones reales:

- **Sistemas GPS:** para calcular rutas eficientes.
- **Redes informáticas:** para enviar datos por los canales adecuados.
- **Logística y transporte:** para reducir costes y tiempos de entrega.
- **Planificación urbana:** para trazar recorridos en mapas o redes.

Por ello, su estudio es fundamental tanto en la Matemática Discreta como en la Informática.

UNIVERSIDAD DE CÓRDOBA

Descripción del algoritmo

El algoritmo de Dijkstra es un método basado en la teoría de grafos. Un grafo representa las relaciones entre elementos mediante:

- **Vértices (o nodos):** los puntos de conexión.
- **Aristas:** las uniones entre nodos.

Grafo ponderado

Es aquel donde a cada arista se le asigna un valor numérico llamado **"peso"** (o coste), que indica la importancia o el coste de transitarla.

UNIVERSIDAD DE CÓRDOBA

Historia y Estrategia

La solución más eficiente a este problema la aportó **Edsger Dijkstra en 1956**.

Su algoritmo opera bajo la condición de que todos los pesos de la red sean positivos y utiliza una estrategia de **"pasos seguros"**:

1. **Inicialización**: Distancia origen $= 0$, resto $= \infty$.
2. **Elección**: Selecciona el nodo no visitado con menor coste acumulado.
3. **Relajación**: Evalúa si pasar por el nodo actual acorta la distancia a sus vecinos.
4. **Cierre**: Marca el nodo como definitivo.

Fundamento matemático

Definición (Grafo ponderado)

Un grafo ponderado es un par (G, w), donde $G = (V, E)$ es un grafo y $w : E \to \mathbb{R}^+$ es una función de peso.

Peso de un camino C

Para un camino $C : v_0 v_1 \ldots v_k$, el peso es:

$$w(C) = \sum_{i=1}^{k} w(v_{i-1}, v_i)$$

Distancia entre vértices

$$d_G(u, v) = \text{mín}\{w(C) \mid C \text{ es un } u\text{-}v \text{ camino}\}$$

Si no existe camino, $d_G(u, v) = +\infty$.

Principio de optimalidad y Relajación

Principio de optimalidad de Bellman
Cualquier subcamino de un camino mínimo es también un camino mínimo.

Operación de relajación
Para cada arista $(u, v) \in E$:

Actualización de distancia

$$\text{si } d[u] + w(u, v) < d[v] \implies d[v] \leftarrow d[u] + w(u, v)$$

Complejidad computacional

El algoritmo requiere pesos no negativos ($w : E \to \mathbb{R}^+$).

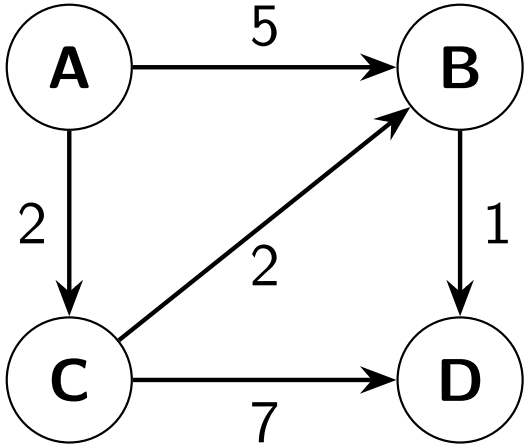
Estructura de datos	**Complejidad temporal**
Lista o Arreglo simple	$O(\|V\|^2)$
Heap binario	$O((\|V\| + \|E\|) \log \|V\|)$
Heap de Fibonacci	$O(\|E\| + \|V\| \log \|V\|)$

Cuadro: Eficiencia según la implementación

Ejemplo: Definición del grafo

Definimos los nodos y sus conexiones:

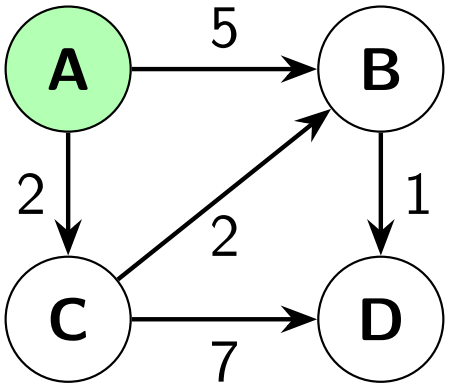
- $(A, B) \to 5$
- $(A, C) \to 2$
- $(B, D) \to 1$
- $(C, B) \to 2$
- $(C, D) \to 7$



Ejemplo: Desarrollo (I)

Paso 1: Seleccionamos Nodo A (Origen)

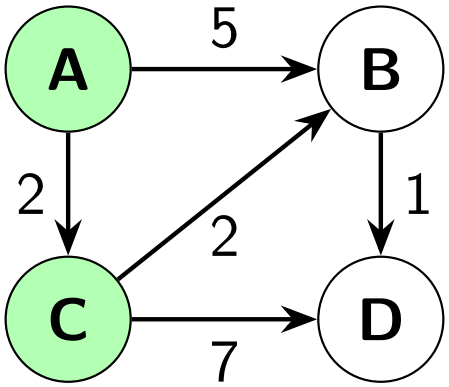
- Distancias iniciales: $A = 0$, resto ∞.
- Vecinos de A: B y C.
- $B : 0 + 5 = 5 < \infty$ (Actualizamos).
- $C : 0 + 2 = 2 < \infty$ (Actualizamos).



Ejemplo: Desarrollo (II)

Paso 2: Seleccionamos Nodo C (Distancia 2)

- Elegimos C porque $2 < 5$ (B).
- Vecinos de C: B y D.
- $B : 2 + 2 = 4$. Como $4 < 5$, **actualizamos B**.
- $D : 2 + 7 = 9 < \infty$ (Actualizamos).



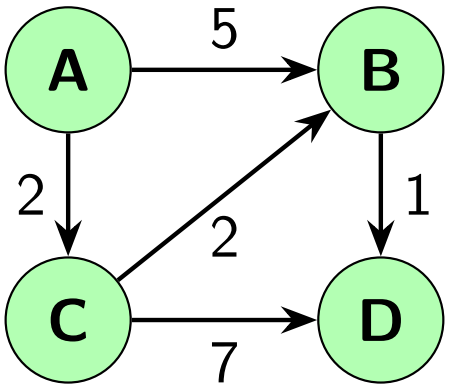
Ejemplo: Desarrollo (III)

Paso 3: Seleccionamos Nodo B (Distancia 4)

- Vecino de B: D.
- $D : 4 + 1 = 5$. Como $5 < 9$, **actualizamos D**.

Paso 4: Seleccionamos Nodo D (Distancia 5)

- No hay más vecinos no visitados. Algoritmo finalizado.



Resumen de Resultados

Nodo	Distancia Mínima	Camino (Predecesores)
A	0	-
B	4	$A \to C \to B$
C	2	$A \to C$
D	5	$A \to C \to B \to D$

Conclusión del ejemplo

La ruta óptima para llegar a D no es la directa, sino pasando por C y B, logrando un coste total de 5 frente a opciones más pesadas.

UNIVERSIDAD DE CÓRDOBA

Pseudocódigo del algoritmo

Función Dijkstra(Grafo, origen)

```
dist[origen] = 0
Para cada vértice v en Grafo:
    Si v != origen:
        dist[v] = infinito
    Añadir v a Q (conjunto no visitados)

Mientras Q no esté vacío:
    u = vértice en Q con mínima dist[u]
    Eliminar u de Q
    Para cada vecino v de u todavía en Q:
        alt = dist[u] + peso(u, v)
        Si alt < dist[v]:
            dist[v] = alt
            prev[v] = u
Devolver dist, prev
```

Explicación del pseudocódigo

El algoritmo calcula la ruta más corta desde un origen hacia todos los demás nodos. Requiere que las distancias sean positivas. **Idea principal:**

Seleccionar siempre el nodo no visitado más cercano al origen y actualizar las distancias de sus vecinos si se halla un camino más corto.

Explicación: Línea por línea I

- `dist[source] = 0`: La distancia del punto de inicio a sí mismo es cero.
- `for each vertex v in Graph`: Recorre todos los nodos de la red para su configuración inicial.
- `dist[v] = infinity`: Al inicio, el algoritmo desconoce cómo llegar al resto de nodos, por lo que les asigna una distancia infinita.
- `add v to Q`: Introduce todos los nodos en un conjunto `Q` (lista de tareas pendientes).

UNIVERSIDAD DE CÓRDOBA

Explicación: Línea por línea II

- `while Q is not empty`: El proceso se repite hasta que no queden nodos por visitar.
- `u = vertex in Q with min dist[u]`: Selecciona el nodo con la menor distancia acumulada. En la primera iteración, siempre será el origen.
- `remove u from Q`: Marca el nodo como visitado eliminándolo de la lista de pendientes.
- `for each neighbor v of u in Q`: Revisa los vecinos de `u` que aún no han sido finalizados.

UNIVERSIDAD DE CÓRDOBA

Explicación: Línea por línea III

- `alt = dist[u] + length(u, v)`: Calcula la longitud de una posible nueva ruta sumando la distancia a `u` más el peso de la arista hasta `v`.
- `if alt < dist[v]`: Es el núcleo del algoritmo; comprueba si esta nueva ruta es mejor que la mejor conocida hasta ahora.
- `dist[v] = alt` y `prev[v] = u`: Si es mejor, actualiza el registro y guarda a `u` como el predecesor óptimo.

UNIVERSIDAD DE CÓRDOBA

Resultado final del algoritmo

- `return dist, prev`: El algoritmo devuelve:
  - `dist`: Distancia mínima del origen a cada punto.
  - `prev`: Estructura para reconstruir la ruta exacta.

Utilidad

Gracias a esto, no solo sabemos cuánto cuesta llegar, sino qué camino exacto debemos seguir.

UNIVERSIDAD DE CÓRDOBA

Simulación del algoritmo

Hemos desarrollado una simulación interactiva en `React.js` para visualizar este proceso:

Acceder a la Simulación

Conclusiones

- El algoritmo de Dijkstra es fundamental para la optimización de rutas.
- Su eficiencia depende de la estructura de datos utilizada (Heaps).
- Es la base de tecnologías que usamos a diario como GPS y protocolos de internet.
- Garantiza la solución óptima siempre que los pesos sean no negativos.

UNIVERSIDAD DE CÓRDOBA

Bibliografía y webgrafía

- **Blog**: Byte, L. (2024). El Algoritmo de Dijkstra. CodigoNautas. https://codigonautas.com/algoritmo-dijkstra-que-es-ejemplo/
- **Video**: Gomez, J. A. (2012). Algoritmo de Dijkstra - Teoría de Grafos. YouTube. https://www.youtube.com/watch?v=fgdCNuGPJnw

UNIVERSIDAD DE CÓRDOBA